Пермский институт (филиал) федерального государственного бюджетного образовательного учреждения высшего образования

«Российский экономический университет имени Г.В. Плеханова»

Кафедра информационных технологий и программирования

Практическая работа #2

Тема работы: Создание и тест

простого калькулятора

Работу выполнил:

Зеленикин Илья Максимович

Группа: ИПО-21

Преподаватель: Берестов

Дмитрий Борисович

Пермь 2026

Оглавление

Цель работы 3
Описание проекта 3
Структура проекта 3
Ход работы 3
1.1 Создание Библиотеки классов 3
2.1 Создание WPF 4
2.2 Логика работы калькулятора 7
2.3 Создание тестов 11
Заключение 13

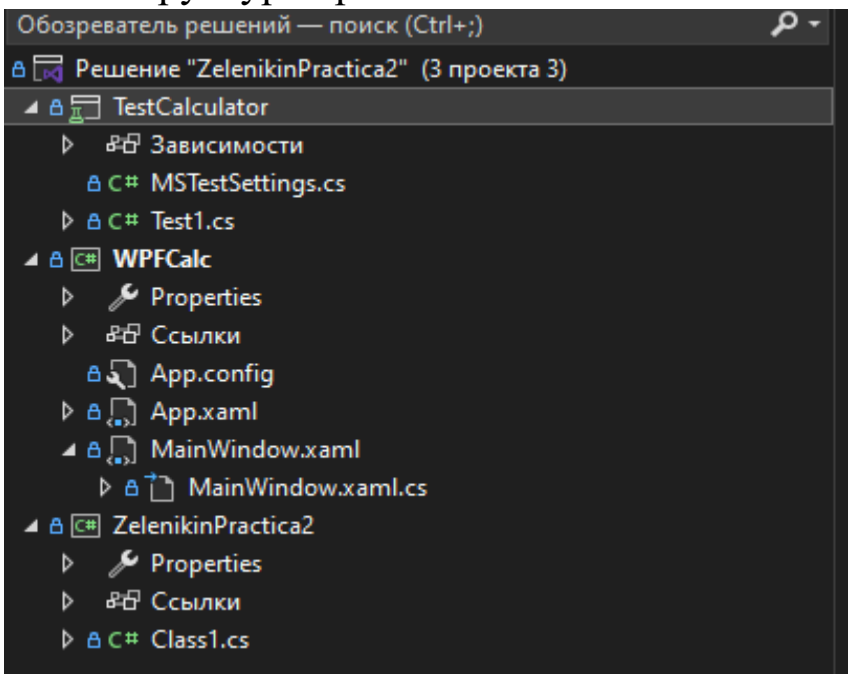
Цель работы

Разработать калькулятор с использованием WPF-технологии и отдельной библиотеки классов, а также реализовать тестирование для проверки корректности функционирования приложения.

Описание проекта

В рамках работы создан проект Calc — приложение-калькулятор на платформе WPF. Программа поддерживает выполнение основных математических операций (сложение, вычитание, умножение, деление), возведение в степень, работу со скобками и десятичными значениями. Графический интерфейс реализован через Grid-разметку с набором кнопок для ввода числовых значений, операторов и команд управления (очистка, десятичная точка, вычисление результата). Для автоматизированной проверки функционала разработан отдельный тестовый проект TestCalc на базе MSTest.

Структура проекта



Ход работы

1.1 Создание Библиотеки классов

На начальном этапе была создана библиотека классов (.NET Framework). После генерации проекта в неё был добавлен программный код,

представленный на рисунке 1.1.

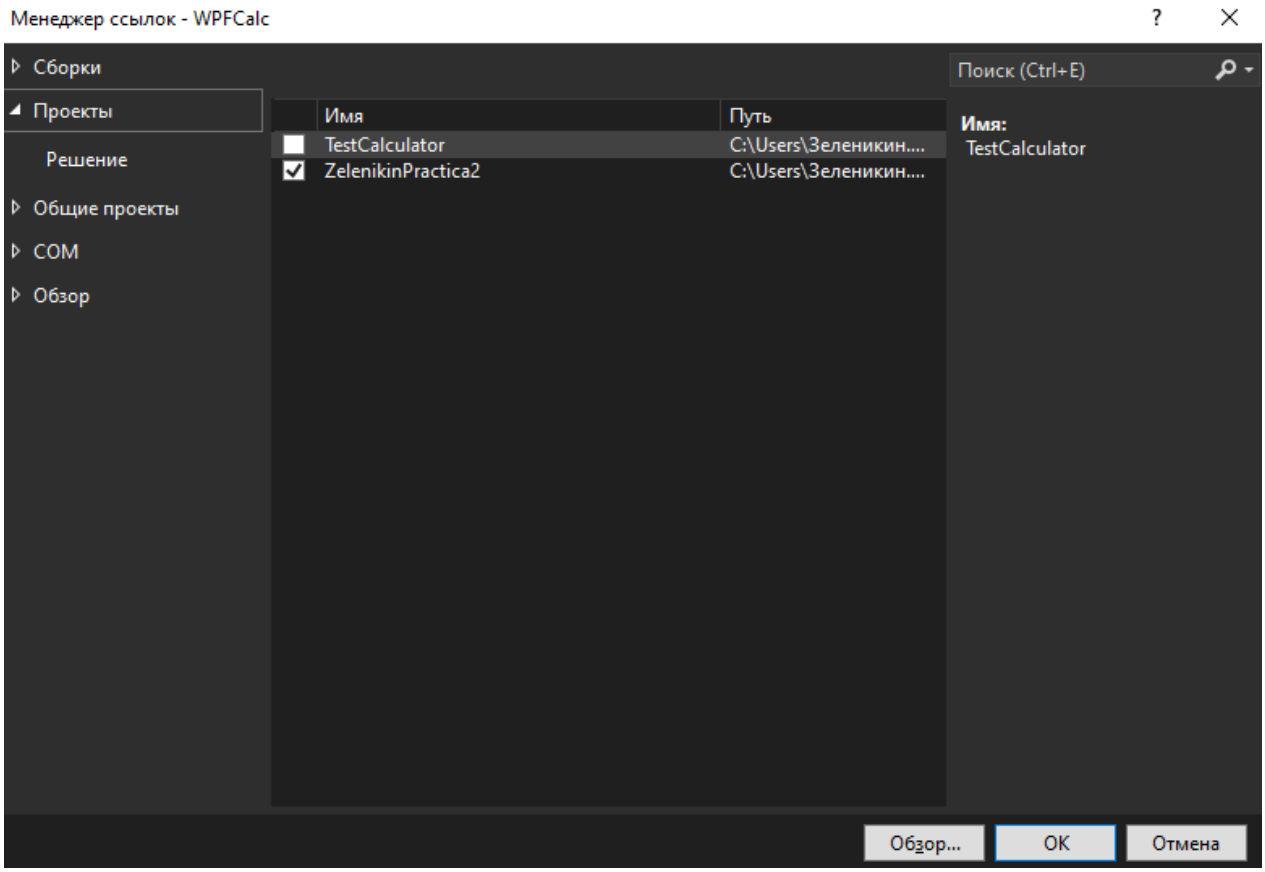
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ZelenikinPractica2
{
    Ссылок: 6
    public class Class1
    {
        Ссылок: 6 | 5/5 passing
        public static dynamic Exicute(dynamic first, char op, dynamic second)
        {
            dynamic result = 0;
            switch (op)
            {
                case '+':
                    result = first + second;
                    break;
                case '-':
                    result = first - second;
                    break;
                case '*':
                    result = first * second;
                    break;
                case '/':
                    result = first / second;
                    break;
                case '^':
                    result = Math.Pow(first, second);
                    break;
                default:
                    result = 0;
                    break;
            }
            return result;
        }
    }
}
```

Рисунок 1.1 – код библиотеки классов

код в одном методе. Код нужен для построения следующих решений по выполнению практики

2.1 Создание WPF

Подключаем библиотеку классов к WPF ссылкой



создаём разметку и размеры окна калькулятора рисунок 2.1

```
    Title="Калькулятор" Height="600" Width="500">
<Grid x:Name="MainLayout">
    <Grid.RowDefinitions>
        <RowDefinition></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
        <RowDefinition Height="90"></RowDefinition>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition></ColumnDefinition>
        <ColumnDefinition></ColumnDefinition>
        <ColumnDefinition></ColumnDefinition>
        <ColumnDefinition></ColumnDefinition>
    </Grid.ColumnDefinitions>
```

Рисунок 2.1 – создание сетки и минимальных размеров окна

Далее с помощью элемента StackPanel была сформирована верхняя панель интерфейса, в которую добавлены два текстовых поля (TextBox). Первое поле отображает введённое выражение, второе — показывает результат вычислений (рисунок 2.2).

Создаём кнопки для калькулятора рисунок 2.3:

```
<Button x:Name="one" Grid.Column="0" Grid.Row="2" Content="1" FontSize="36"/>
<Button x:Name="two" Grid.Column="1" Grid.Row="2" Content="2" FontSize="36"/>
<Button x:Name="free" Grid.Column="2" Grid.Row="2" Content="3" FontSize="36"/>
<Button x:Name="four" Grid.Column="0" Grid.Row="3" Content="4" FontSize="36"/>
<Button x:Name="five" Grid.Column="1" Grid.Row="3" Content="5" FontSize="36"/>
<Button x:Name="six" Grid.Column="2" Grid.Row="3" Content="6" FontSize="36"/>
<Button x:Name="seven" Grid.Column="0" Grid.Row="4" Content="7" FontSize="36"/>
<Button x:Name="eight" Grid.Column="1" Grid.Row="4" Content="8" FontSize="36"/>
<Button x:Name="nine" Grid.Column="2" Grid.Row="4" Content="9" FontSize="36"/>
<Button x:Name="C" Grid.Column="0" Grid.Row="5" Content="C" FontSize="36"/>
<Button x:Name="null" Grid.Column="1" Grid.Row="5" Content="0" FontSize="36"/>
<Button x:Name="dot" Grid.Column="2" Grid.Row="5" Content="." FontSize="36"/>
<Button x:Name="smooth" Grid.Column="3" Grid.Row="5" Content="=" FontSize="36"/>
<Button x:Name="plus" Grid.Column="3" Grid.Row="4" Content="+" FontSize="36"/>
<Button x:Name="minus" Grid.Column="3" Grid.Row="3" Content="-" FontSize="36"/>
<Button x:Name="multiply" Grid.Column="3" Grid.Row="2" Content="*" FontSize="36"/>
<Button x:Name="division" Grid.Column="3" Grid.Row="1" Content="/" FontSize="36"/>
<Button x:Name="RaistToPower" Grid.Column="2" Grid.Row="1" Content="^" FontSize="36"/>
<Button x:Name="RightParenthesis" Grid.Column="1" Grid.Row="1" Content=")" FontSize="36"/>
<Button x:Name="LeftParenthesis" Grid.Column="0" Grid.Row="1" Content="(" FontSize="36"/>
```

Рисунок 2.3 – кнопки калькулятора

Результат как выглядит калькулятор в конструкторе рисунок 2.4

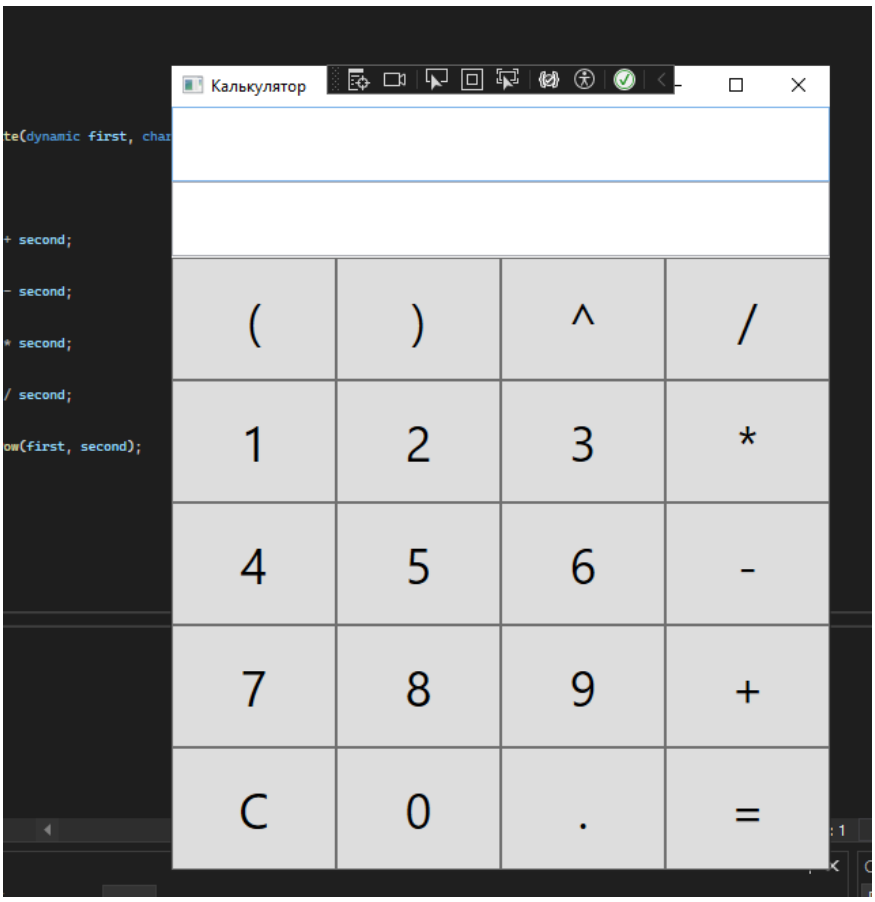


Рисунок 2.4 – калькулятор вид в конструкторе

2.2 Логика работы калькулятора

Файл MainWindow.xaml.cs отвечает за взаимодействие пользователя с интерфейсом калькулятора. Весь код можно разделить на две логические части:

- Код в MainWindow

```
namespace WPFCalc
{
    Ссылок: 2
    public partial class MainWindow : Window
    {
        Ссылок: 0
        public MainWindow()
        {
            InitializeComponent();
            tbCommand.Clear();
            foreach (UIElement element in MainLayout.Children)
            {
                if (element is Button)
                {
                    ((Button)element).Click += Button_Click;
                }
            }
        }
```

InitializeComponent() - инициализирует визуальные элементы файла MainWindow.xaml (кнопки, сетку, текстовые поля).

tbCommand.Clear() - очищает поле ввода при запуске, обеспечивая пустой экран при открытии калькулятора.

Автоматическая подписка на события:

1. Программа выполняет обход всех элементов в контейнере MainLayout
2. При обнаружении элементы типа (Button), к ней автоматически привязывается метод Button_Click.
3. Преимущество: любая новая кнопка, добавленная в интерфейс, будет автоматически обрабатываться без модификации кода

– **Код в методе Button_Click**

Метод обрабатывает нажатия всех кнопок и различные действия в зависимости содержимого кнопки.

Получение текста из нажатой кнопки:

```
string TextButton = ((Button)e.OriginalSource).Content.ToString();
```

Определяется какая кнопка нажата и извлекается из неё текст.

```
if (TextButton == "C")
{
    tbCommandHistory.Clear();
    tbCommand.Clear();
}
```

Обработка кнопки =

a) Получения выражения и поиск операторов

```
string expr = tbCommand.Text;
char[] ops = { '+', '-', '*', '/', '^' };
int pos = -1;
char op = ' ';
for (int i = 0; i < expr.Length; i++)
{
    if (Array.IndexOf(ops, expr[i]) >= 0)
    {
        pos = i;
        op = expr[i];
        break;
    }
}
```

Программа считывает выражение из поля ввода и последовательно ищет первый арифметический оператор (+, -, *, /, ^), запоминая его позицию и сам знак.

b) Разделение на операнды и преобразование:

```
if (pos != -1)
{
    string num1Text = expr.Substring(0, pos);
    string num2Text = expr.Substring(pos + 1);

    double num1 = double.Parse(num1Text);
    double num2 = double.Parse(num2Text);
```

выражение разделяется на две части: первое число (до оператора) и второе число (после оператора). Обе строки преобразуются в значения типа double

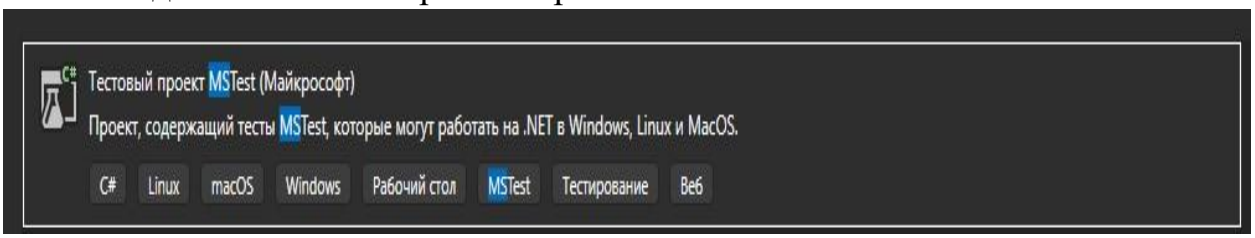
c) Вызов вычислений и вывод результата:

```
dynamic res = Class1.Exicute(num1, op, num2);

tbCommandHistory.Text = $"{expr} =";
tbCommand.Text = res.ToString();
```

Числа и оператор передаются в метод Class1.Exicute, который выполняет математическую операцию. Результат отображается в основном поле, а само выражение с знаком = сохраняется в поле истории.

2.3 Создание тестов

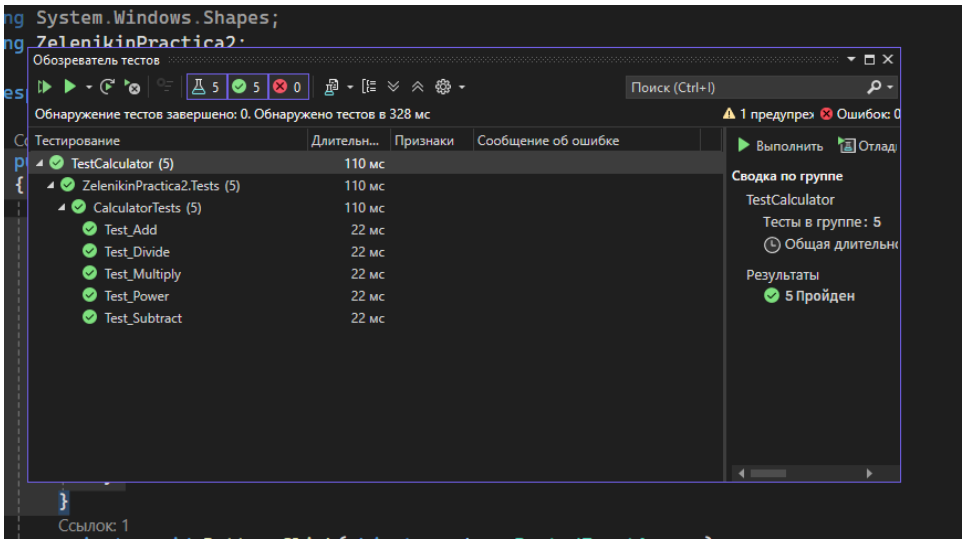
Создаём в этом же проекте проект с тестами:



Затем также подключаем ссылкой к библиотеке, и пишем код для проверки калькулятора:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using ZelenikinPractica2;

namespace ZelenikinPractica2.Tests
{
    [TestClass]
    Ссылок: 0
    public class CalculatorTests
    {
        [TestMethod]
        Ссылок: 0
        public void Test_Add()
        {
            dynamic result = Class1.Exicute(1, '+', 9);
            Assert.AreEqual(10, result);
        }

        [TestMethod]
        Ссылок: 0
        public void Test_Subtract()
        {
            dynamic result = Class1.Exicute(15, '-', 8);
            Assert.AreEqual(7, result);
        }

        [TestMethod]
        Ссылок: 0
        public void Test_Multiply()
        {
            dynamic result = Class1.Exicute(8, '*', 7);
            Assert.AreEqual(56, result);
        }

        [TestMethod]
        Ссылок: 0
        public void Test_Divide()
        {
            dynamic result = Class1.Exicute(25, '/', 5);
            Assert.AreEqual(5, result);
        }

        [TestMethod]
        Ссылок: 0
        public void Test_Power()
        {
            dynamic result = Class1.Exicute(2, '^', 3);
            Assert.AreEqual(8, result);
        }
    }
}
```

После запуска все тесты успешно прошли:



Заключение

В результате выполнения работы было разработано WPF-приложение «Калькулятор» с поддержкой базовых арифметических функций. Архитектура проекта построена по модульному принципу: логика вычислений вынесена в отдельную библиотеку, что облегчает тестирование и дальнейшее расширение функционала. Для верификации корректности работы создан тестовый проект на платформе MSTest. Обработка всех кнопок реализована через единый метод-обработчик, что обеспечивает компактность и гибкость кодовой базы.